LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 1

ANDROID BASICS WITH COMPOSE

Oleh:

Sebastian Abe Santoso NIM. 2410817210002

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 1

Laporan Praktikum Pemrograman Mobile Modul 1: Android Basic with Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Sebastian Abe Santoso
NIM : 2410817210002

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom. NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN i
DAFTAR ISI ii
DAFTAR GAMBAR iii
DAFTAR TABEL iv
SOAL PRAKTIKUM 1
A. Source Code 4
B. Output Program 12
C. Pembahasan 15
D. Essay 18
E. Tautan Git 25

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi 1
Gambar 2. Tampilan Dadu Setelah Di-Roll 2
Gambar 3. Tampilan Roll Dadu Double 2
Gambar 4. Screenshot Hasil Jawaban Soal XML Awal 12
Gambar 5. Screenshot Hasil Jawaban Soal XML Belum Beruntung 12
Gambar 6. Screenshot Hasil Jawaban Soal XML Dadu Double 13
Gambar 7. Screenshot Hasil Jawaban Soal Compose Awal 13
Gambar 8. Screenshot Hasil Jawaban Soal Compose Belum Beruntung 14
Gambar 9. Screenshot Hasil Jawaban Soal Compose Dadu Double 14

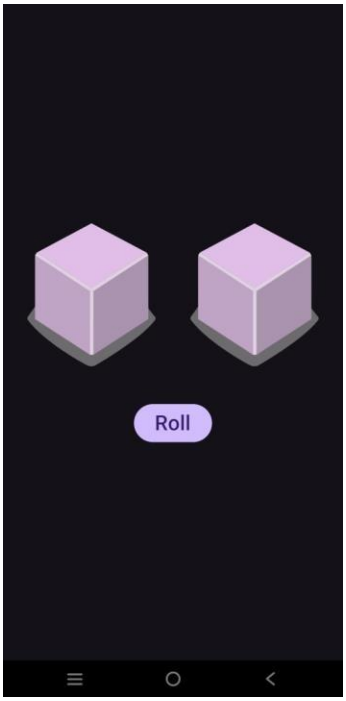
DAFTAR TABEL

Tabel 1. Source Code Main Activity.kt XML 4
Tabel 2. Source Code Main Activity.kt Compose 5
Tabel 3. Source Code activity_main.xml 9

SOAL 1

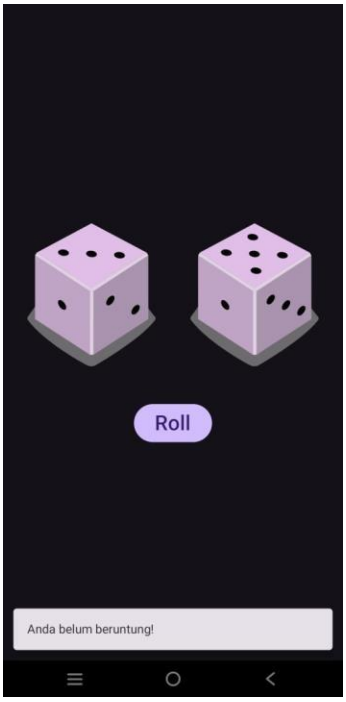
Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



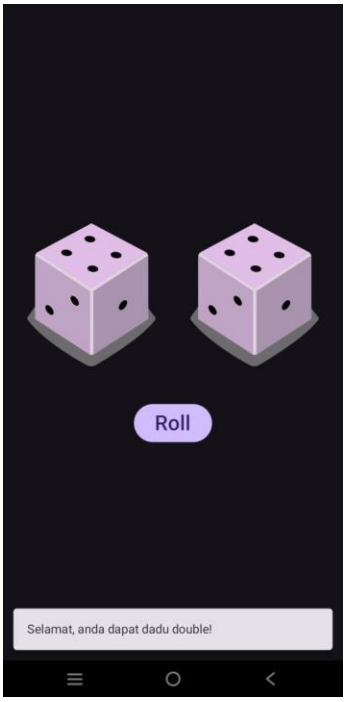
Gambar 1. Tampilan Awal Aplikasi

2. Tampilan Setelah user menekan tombol “Roll” maka masing-masing dadu akan memperlihatkan sisi dadunya dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2, maka aplikasi akan menampilkan pesan “Anda belum beruntung!” seperti yang dapat dilihat pada Gambar 2.



Gambar 2. Tampilan Dadu Setelah Di-Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat, anda dapat dadu double!” seperti yang dapat dilihat pada Gambar 3.



Gambar 3. Tampilan Roll Dadu Double

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam **folder Modul 1 dalam bentuk Project.** Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat didownload pada link berikut: https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp= sharing

Additional:

ESSAY

1. Riset dan *cite* **artikel/jurnal/buku** mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatkan opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan. **

2. Apa yang membedakan OOP pada kotlin dengan OOP pada java? *

3. Sebutkan, dan jelaskan mengenai SOLID Principles *

4. Apa perbedaan antara Class dan Data Class pada Kotlin ?

**** = 2 sumber**

*** = 1 sumber**

Gunakanlah sumber primary artikel/jurnal/buku, web hanya sebagai penunjang opini/argumen

A. Source Code

Tabel 1. Source Code Main Activity.kt XML

No	Main Activity.kt (XML)
1	package com.example.mydiceapplicationxml
2	
3	import android.os.Bundle
4	import android.view.View
5	import android.widget.Button
6	import android.widget.ImageView
7	import android.widget.TextView
8	import androidx.activity.enableEdgeToEdge
9	import androidx.appcompat.app.AppCompatActivity
10	import androidx.core.view.ViewCompat
11	import androidx.core.view.WindowInsetsCompat
12	
13	class MainActivity : AppCompatActivity() {
14	override fun onCreate(savedInstanceState: Bundle?) {
15	super.onCreate(savedInstanceState)
16	enableEdgeToEdge()
17	setContentView(R.layout.activity_main)
18	val dice1 = findViewById<ImageView>(R.id.dice1)
19	val dice2 = findViewById<ImageView>(R.id.dice2)
20	val button = findViewById<Button>(R.id.rollbtn)
21	val text = findViewById<TextView>(R.id.resulttxt)
22	
23	button.setOnClickListener {
24	val result1 = (1..6).random()
25	val result2 = (1..6).random()
26	
27	dice1.setImageResource(setDiceImage(result1))
28	dice2.setImageResource(setDiceImage(result2))
29	
30	text.visibility = View.VISIBLE
31	
32	if (result1 != 0 && result2 != 0) {
33	if (result1 == result2){
34	text.text = "Selamat, anda dapat dadu double!"
35	} else {
36	text.text = "Anda belum beruntung!"
37	}
38	}
39	}
40	

```
ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
            insets
        }
    }
}


fun setDiceImage(result: Int): Int {
    return when(result) {
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        6 -> R.drawable.dice_6
        else -> R.drawable.dice_1
    }
}
```

Tabel 2. Source Code Main Activity.kt Compose

No	**Main Activity.kt (Compose)**
1	`package com.example.mydiceapplication`
2	`import androidx.compose.runtime.getValue`
3	`import android.os.Bundle`
4	`import androidx.activity.ComponentActivity`
5	`import androidx.activity.compose.setContent`
6	`import androidx.activity.enableEdgeToEdge`
7	`import androidx.compose.foundation.Image`
8	`import androidx.compose.foundation.background`
9	`import androidx.compose.foundation.layout.Arrangement`
10	`import androidx.compose.foundation.layout.Box`
11	`import androidx.compose.foundation.layout.Column`
12	`import androidx.compose.foundation.layout.Row`
13	`import androidx.compose.foundation.layout.Spacer`
14	`import androidx.compose.foundation.layout.fillMaxSize`
15	`import androidx.compose.foundation.layout.fillMaxWidth`
16	`import androidx.compose.foundation.layout.height`
17	`import androidx.compose.foundation.layout.padding`
18	`import androidx.compose.material3.Button`
19	`import androidx.compose.material3.ButtonDefaults`
20	`import androidx.compose.material3.Scaffold`

```
21 import androidx.compose.material3.Text
22 import androidx.compose.runtime.Composable
23 import androidx.compose.runtime.mutableIntStateOf
24 import androidx.compose.runtime.remember
25 import androidx.compose.runtime.setValue
26 import androidx.compose.ui.Alignment
27 import androidx.compose.ui.Modifier
28 import androidx.compose.ui.graphics.Color
29 import androidx.compose.ui.res.painterResource
30 import androidx.compose.ui.tooling.preview.Preview
31 import androidx.compose.ui.unit.dp
32 import androidx.compose.ui.unit.sp
33 import
   com.example.mydiceapplication.ui.theme.MyDiceApplicati
   onTheme
34
35 class MainActivity : ComponentActivity() {
36     override fun onCreate(savedInstanceState: Bundle?)
37 {
38         super.onCreate(savedInstanceState)
39         enableEdgeToEdge()
40         setContent {
41             MyDiceApplicationTheme {
42                 Scaffold(
43                     modifier = Modifier.fillMaxSize(),
44                     containerColor = Color.Black
45                 ) { innerPadding ->
46                     DiceRollerApp(
47                         modifier =
   Modifier.padding(innerPadding)
48                     )
49                 }
50             }
51         }
52     }
53 }
54
55 fun getDiceImage(result: Int): Int{
56     return when (result){
57         1 -> R.drawable.dice_1
58         2 -> R.drawable.dice_2
59         3 -> R.drawable.dice_3
60         4 -> R.drawable.dice_4
61         5 -> R.drawable.dice_5
62         6 -> R.drawable.dice_6
63         else -> R.drawable.dice_0
64     }
```

```
}

@Composable
fun DiceRollerApp(modifier: Modifier = Modifier){
    var result1 by remember { mutableIntStateOf(0)}
    var result2 by remember { mutableIntStateOf(0)}

    val imageResource1 = getDiceImage(result1)

    val imageResource2 = getDiceImage(result2)

    Column (
        modifier = modifier
            .fillMaxSize()
            .background(Color.Black),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Spacer(modifier = Modifier.weight(1f))
        Row (
            horizontalArrangement = Arrangement.Center
        ) {
            Image(
                painter = painterResource(imageResource1),
                contentDescription = result1.toString()
            )

            Image(
                painter = painterResource(imageResource2),
                contentDescription = result2.toString()
            )
        }

        Button(
            onClick = {
            result1 = (1..6).random()
            result2 = (1..6).random()
        },
            colors = ButtonDefaults.buttonColors(
                containerColor = Color.hsl(261f, 0.8f, 0.8f),
                contentColor = Color.Black,
```

```
            )
        ) {
            Text(
                text = "Roll",
                fontSize = 20.sp
            )
        }

        Spacer(modifier = Modifier.weight(1f))

        Column(
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Bottom
        ) {
            if (result1 != 0 && result2 != 0) {
                if (result1 == result2) {
                    Box(
                        modifier = Modifier
                            .background(
                                color = Color(0xFFE1D9E1),
                            )
                            .fillMaxWidth(0.95f)
                            .padding(10.dp)
                    ){
                        Text(
                            text = "Selamat, anda dapat dadu double",
                            color = Color.Black
                        )
                    }
                } else {
                    Box(
                        modifier = Modifier
                            .background(
                                color = Color(0xFFE1D9E1),
                            )
                            .fillMaxWidth(0.95f)
                            .padding(10.dp)
                    ){
                        Text(
                            text = "Anda belum beruntung!",
                            color = Color.Black
                        )
```

```
                    }
                }
            }
        }
        Spacer(modifier = modifier.height(20.dp))
    }
}

@Preview(showBackground = true)
@Composable
fun PreviewDiceRollerApp(){
    MyDiceApplicationTheme { DiceRollerApp() }
}
```

Tabel 3. Source Code activity_main.xml

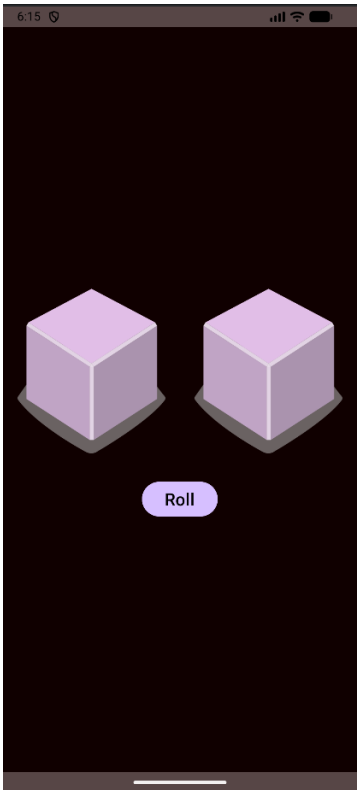
No | **activity_main.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/androi
d"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#000000"
    android:backgroundTint="#574646"
    tools:context=".MainActivity">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:background="#100000"
        android:gravity="fill"
        android:orientation="vertical"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="525dp"
            android:gravity="center"
            android:orientation="horizontal">

```

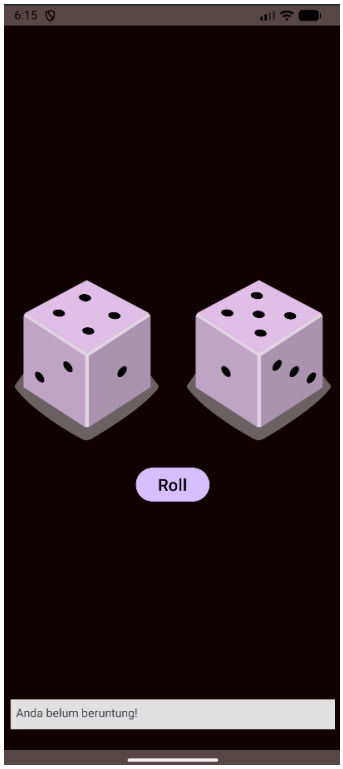
```
            <ImageView
                android:id="@+id/dice1"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_gravity="bottom"
                android:layout_weight="1"
                app:srcCompat="@drawable/dice_0" />

            <ImageView
                android:id="@+id/dice2"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_gravity="bottom"
                android:layout_weight="1"
                app:srcCompat="@drawable/dice_0" />

        </LinearLayout>

        <LinearLayout
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_gravity="center"
            android:gravity="center"
            android:orientation="horizontal">

            <Button
                android:id="@+id/rollbtn"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:backgroundTint="#d6bfff"
                android:text="Roll"
                android:textColor="#000000"
                android:textColorHighlight="#000000"
                android:textColorHint="#050505"
                android:textSize="20sp" />
        </LinearLayout>

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:gravity="bottom"
            android:orientation="horizontal">

            <TextView
                android:id="@+id/resulttxt"
                android:layout_width="wrap_content"
```

```
                android:layout_height="36dp"
                android:layout_gravity="bottom|center"
                android:layout_marginStart="12dp"
                android:layout_marginEnd="12dp"
                android:layout_marginBottom="12dp"
                android:layout_weight="1"
                android:background="#E0E0E0"
                android:padding="6dp"
                android:visibility="invisible" />

        </LinearLayout>
    </LinearLayout>

</androidx.constraintlayout.widget.ConstraintLayout>
```

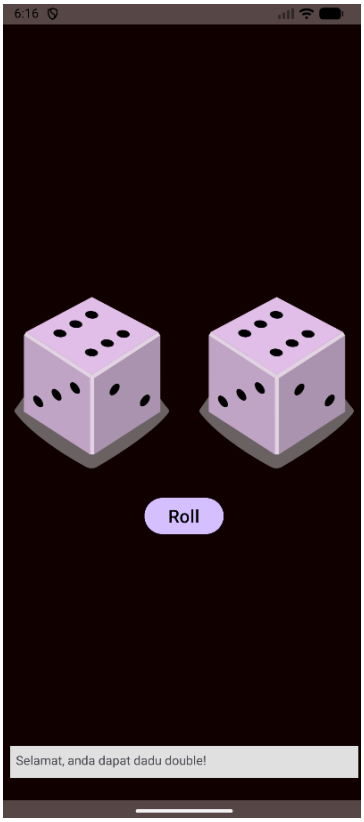
B. Output Program



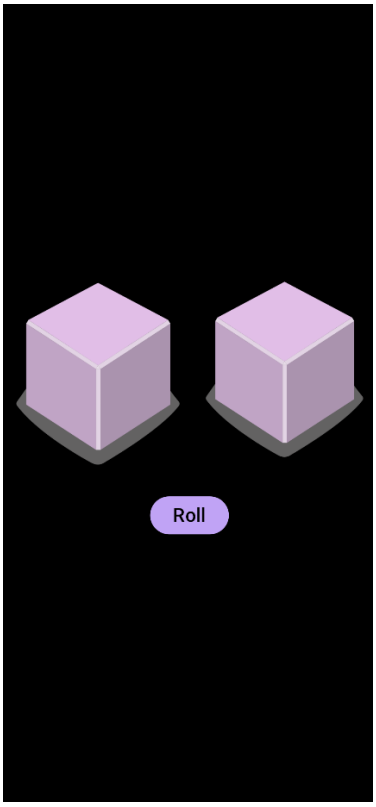
Gambar 4. Screenshot Hasil Jawaban Soal XML Awal



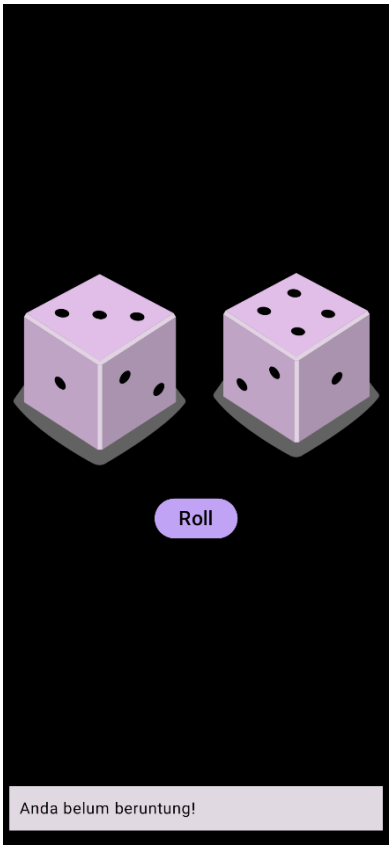
Gambar 5. Screenshot Hasil Jawaban Soal XML Belum Beruntung



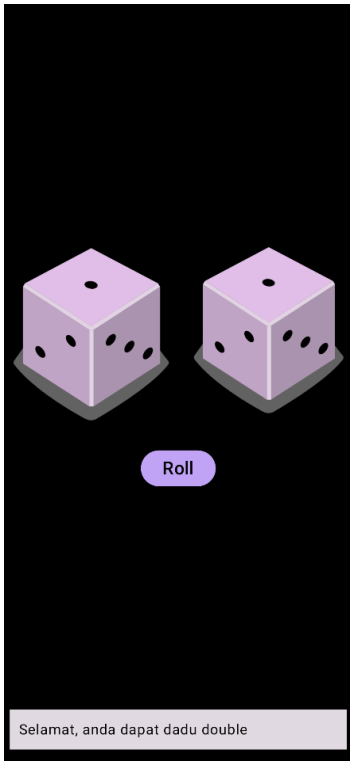
Gambar 6. Screenshot Hasil Jawaban Soal XML Dadu Double



Gambar 7. Screenshot Hasil Jawaban Soal Compose Awal



Gambar 8. Screenshot Hasil Jawaban Soal Compose Belum Beruntung



Gambar 9. Screenshot Hasil Jawaban Soal Compose Dadu Double

C. Pembahasan

MainActivity.kt XML

Kode ini berguna untuk melempar dua dadu, menampilkan hasil dalam bentuk gambar, dan menampilkan hasil "belum beruntung" jika angka beda, dan "double" jika angka sama, file ini secara utama berguna sebagai entry point layer, dan UI-nya berada di file lain (`activity_main.xml`), mulai dari package, kemudian import library sesuai kebutuhan.

Mulai dari `class MainActivity : AppCompatActivity()`, yaitu `class MainActivity` yang dibuat untuk inherit dari `AppCompatActivity()` yang berguna untuk mendapatkan lifecycle method seperti `onCreate, onStart`, dll.

Kemudian terdapat lifecycle method (`override fun onCreate(savedInstanceState: Bundle?)`) yang dijalankan pertama kali saat `Activity` pertama dibuat, digunakan untuk setup awal seperti UI, data, listener, dll.

Selanjutnya, terdapat `enableEdgeToEdge()` yang berguna agar UI bisa fullscreen, `setContentView(R.layout.activity_main)` yang menghubungkan layout XML (`activity_main.xml`) dengan `Activity`, `findViewById<ImageView>()` yang mengambil komponen UI dari XML agar bisa bisa dikontrol di Kotlin.

Lalu, terdapat `button.setOnClickListener{}` yang berguna untuk mendeteksi klik dari user, dan berisi:

1. `(1..6).random()`, menghasilkan angka acak dari range 1 - 6 sebagai simulasi dadu.
2. `setImageResource(...)`, mengubah gambar pada ImageView, dalam kode digunakan untuk mengubah gambar dadu sesuai dengan nomor acak di 1.
3. `text.visibility = View.VISIBLE`, mengatur apakah komponen terlihat atau tidak, dalam kode digunakan agar menampilkan hasil setelah tombol ditekan.
4. `If (result1 == resullt2)`, percabangan logika untuk menentukan kondisi, yaitu menentunkan teks yang akan disampaikan, jika sama > dadu double, dan tidak > belum beruntung.

Kemudian, terdapat `ViewCompat.setOnApplyWindowInsetsListener()` yang berguna untuk mengambil ukuran system bar, dan menambahkan padding ke view ut sama, sehingga layout dengan status bar dan navigation bar tidak tertutup.

Terakhir, terdapat `fun setDiceImage(result: Int): Int` yaitu fungsi yang berguna untuk *mapping* angka ke gambar dadu, dengan `when()`, yaitu switch di Kotlin yang berguna untuk menentukan gambar berdasarkan nilai.

activity_main.xml

Layout XML ini berguna untuk mendefinisikan tampilan visual dari layar utama aplikasi, berisi dua gambar dadu, tombol roll, dan teks hasil, sebagai bagian UI dari `MainActivity.kt`, dimulai dengan deklarasi `<?xml version="1.0" encoding="utf-8"?>` yang menandakan file XML dengan format karakter UTF-8.

Mulai dari `<androidx.constraintlayout.widget.ConstraintLayout>` sebagai *root layout* yang menampung seluruh komponen UI, dengan `match_parent` pada `width` dan `height` agar memenuhi seluruh layar, serta `android:id="@+id/main"` agar bisa diakses dari Kotlin.

Kemudian, `<LinearLayout>` pertama sebagai *container* utama dengan `orientation="vertical"` sehingga isi disusun dari atas ke bawah, dan *constraint* ke semua sisi *parent* agar mengisi seluruh area `ConstraintLayout`.

Di dalamnya terdapat `<LinearLayout>` kedua khusus untuk area dadu, dengan `orientation="horizontal"` agar dua dadu berdampingan, `layout_height="525dp"` sebagai tinggi tetap, dan `gravity="center"` agar isi berada di tengah.

Di dalam `LinearLayout dadu`, terdapat dua `<ImageView>` yaitu `dice1` dan `dice2`, masing-masing dengan `layout_weight="1"` agar ruang terbagi merata, `layout_gravity="bottom"` agar gambar berada di bawah *container*, dan `app:srcCompat="@drawable/dice_0"` sebagai gambar awal.

Lalu, `<LinearLayout>` ketiga sebagai wadah tombol, dengan `wrap_content` dan `layout_gravity="center"` agar tombol berada di tengah layar tepat di bawah dadu.

Di dalamnya terdapat <Button> dengan android:id="@+id/rollbtn", android:text="Roll" sebagai label, android:backgroundTint="#d6bfff" sebagai warna, dan android:textSize="20sp" sebagai ukuran teks.

Terakhir, <LinearLayout> keempat sebagai wadah hasil dengan gravity="bottom" agar isi menempel ke bawah layar, yang berisi <TextView> dengan android:id="@+id/resulttxt", android:visibility="invisible" agar tidak terlihat saat awal aplikasi, dan android:background="#E0E0E0" sebagai warna latar teks hasil.

MainActivity.kt Compose

Kode ini adalah implementasi aplikasi lempar dadu seperti sebelumnya, tetapi menggunakan pendekatan modern Android yaitu Jetpack Compose. Berbeda dengan versi XML sebelumnya, seluruh tampilan dan logika UI ditulis langsung dalam Kotlin menggunakan fungsi @Composable. Aplikasi ini berfungsi untuk menghasilkan dua angka acak (1 - 6), menampilkan gambar dadu sesuai angka tersebut, dan menampilkan hasil berupa teks “double” jika kedua angka sama atau “belum beruntung” jika berbeda. File ini juga berperan sebagai entry point layar karena menggunakan MainActivity, sekaligus mengatur UI dan state secara langsung dalam satu file, karena itu sangat tidak disarankan untuk tidak membuat kodenya seperti ini, lebih baik dipisah-pisah.

Pada bagian awal, Android menjalankan MainActivity sebagai entry point aplikasi. Di dalam onCreate, terjadi pemanggilan setContent, yang berarti sistem tidak lagi menggunakan XML, tetapi langsung merender UI menggunakan Compose. Hal ini terjadi karena Compose bekerja dengan konsep deklaratif, sehingga UI didefinisikan sebagai fungsi dinamis.

Selanjutnya, UI dibungkus dalam theme (MyDiceApplicationTheme) dan Scaffold. Pada bagian ini terjadi pengaturan struktur dasar layar seperti background dan area konten, karena Scaffold berfungsi sebagai kerangka.

Kemudian, fungsi DiceRollerApp dipanggil. Pada bagian ini terjadi inisialisasi state menggunakan remember dan mutableIntStateOf.

Setelah *state* dibuat, sistem menentukan gambar dadu dengan memanggil `getDiceImage`. Pada bagian ini terjadi *mapping* angka ke *resource* gambar, karena angka *random* tidak bisa langsung ditampilkan tanpa diubah menjadi gambar.

Masuk ke bagian UI utama, `Column` digunakan untuk menyusun elemen secara vertikal. Di dalamnya terdapat `Row` yang menampilkan dua gambar dadu. Pada bagian ini terjadi *rendering* gambar berdasarkan *state* saat ini, karena `painterResource` mengambil *resource* sesuai nilai *result*.

Kemudian terdapat `Button`. Pada bagian ini terjadi interaksi user, karena saat tombol ditekan (`onClick`), kedua nilai dadu diubah menjadi angka random. Perubahan ini menyebabkan state berubah, dan karena Compose bersifat reaktif, UI akan otomatis diperbarui.

Setelah itu, terdapat bagian kondisi hasil. Pada bagian ini terjadi pengecekan apakah kedua dadu sudah memiliki nilai dan apakah nilainya sama. Jika sama, maka ditampilkan teks "Selamat, anda dapat dadu double", jika tidak maka "Anda belum beruntung".

Terakhir, `Spacer` digunakan untuk mengatur jarak antar elemen agar layout terlihat rapi.

D. Essay

1. Riset dan cite artikel/jurnal/buku mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatkan opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan. **

"Functional code can be more elegant and succinct when compared to its imperative counterpart: Instead of mutating variables and relying on loops and conditional branching, working with functions as values gives you much more power of abstraction." [1] - *Chapter 1.2.3*

Di sini dapat dipahami bahwa fungsional (bagian dari deklaratif) bisa dibilang lebih elegan dari imperative. Daripada memutasi variable dan bergantung kepada loop dan kondisional, fungsi memberi tingkat abstraksi besar yang penting.

"You won't be able to write this code sequentially in an imperative fashion, one call after another, without blocking the underlying threads. With callbacks or reactive streams such simple consecutive logic becomes much more complicated." [1] - Chapter 1.2.4

Di sini juga penulisnya menekankan bahwa imperatif akan sangat kompleks jika ada operasi *async*, karena tidak dapat menunggu hasil tanpa menghentikan thread.

"Google's Jetpack Compose toolkit (https://developer.android.com/jetpack/compose) for building native user interfaces for Android is also designed for Kotlin from the ground up. It embraces Kotlin's language features, and gives you the ability to write less, simpler, and easier-to-maintain code when building the UI of your mobile applications." [1] - Chapter 1.3.2

Kutipan ini menjelaskan bahwa pendekatan fungsional membuat penulisan kode UI menjadi lebih sedikit, lebih sederhana, dan lebih mudah untuk dipelihara karena UI direpresentasikan sebagai fungsi dari *state*.

"....imperative languages, in which the focus is on how the computer should do it." [2] - Chapter 1.2

Kalimat ini memberi tahu bahwa pada bahasa imperatif, penulisan program berfokus pada langkah-langkah tentang bagaimana suatu tugas harus dilakukan oleh komputer. Konsep ini juga berlaku dalam UI, di mana developer harus secara spesifik mengatur setiap perubahan tampilan.

"In a similar vein, the lack of first-class subroutines in most imperative languages reflects in large part the desire to avoid heap allocation, which would be needed for local variables with unlimited extent" [2] - Chapter 3.6.3

Kutipan tersebut menjelaskan bahwa banyak bahasa imperatif tidak mendukung subrutin sebagai *first-class* untuk menghindari penggunaan memori heap. Dalam konteks ini, desain bahasa imperatif cenderung lebih memilih efisiensi dan kontrol terhadap memori dibandingkan fleksibilitas seperti yang ditemukan pada pendekatan fungsional. Saya sendiri tidak ahli dalam programming, sehingga cenderung lebih memilih fleksibilitas fungsional dari fine-control nya imperatif.

"...To make functional programming practical, functional languages provide a number of features that are often missing in imperative languages, including: First-class function values and higher-order functions, Extensive polymorphism, List types and operators Structured function returns, Constructors (aggregates) for structured objects, Garbage collection" [2] - Chapter 10.2

Di sini dijelaskan bahwa agar pemrograman fungsional dapat digunakan secara praktis, berbagai fitur penting disediakan yang umumnya tidak dimiliki oleh bahasa imperatif. Fitur-fitur tersebut adalah fungsi sebagai *first-class citizen* dan *higher-order functions*, polymorphism, tipe data list, dukungan konstruksi objek terstruktur, dan *garbage collection*. Fitur-fitur tersebut sangat berguna sehingga membuat functional programming lebih praktikal dalam beberapa aspek dari imperatif.

"Many programmers, probably most, who have written significant amounts of software in both imperative and functional styles find the latter more aesthetically appealing. Moreover experience with a variety of large commercial projects (see the Bibliographic Notes at the end of the chapter) suggests that the absence of side effects makes functional programs significantly easier to write, debug, and maintain than their imperative counterparts." [2] - Chapter 10.2

Bagian di atas menjelaskan bahwa banyak programmer yang telah berpengalaman menggunakan kedua pendekatan, imperatif dan fungsional, cenderung menganggap pendekatan fungsional lebih menarik secara estetika. Selain itu, berdasarkan pengalaman dalam berbagai proyek skala besar, minimnya *side effects* pada pemrograman fungsional membuat program menjadi lebih mudah untuk ditulis, debug, dan dipelihara. Hal ini berbeda dengan pendekatan imperatif, di mana perubahan state yang tersembunyi sering kali menambah kompleksitas dan potensi bug dalam pengembangan perangkat lunak.

"When you design UI interactions, you probably think about how the UI changes in response to user actions. In imperative programming, You have to write the exact instructions to manipulate the UI depending on what just happened" [3]

Di sini dijelaskan bahwa dalam merancang interaksi UI, developer biasanya memikirkan bagaimana tampilan akan berubah sebagai respons terhadap aksi pengguna. Pada pendekatan imperatif, perubahan tersebut harus ditangani dengan menuliskan instruksi secara eksplisit, yaitu bagaimana UI harus dimanipulasi untuk setiap kejadian yang terjadi. Artinya, setiap perubahan state harus diikuti dengan langkah-langkah detail untuk memperbarui elemen UI, sehingga pendekatan ini cenderung lebih kompleks dan membutuhkan pengelolaan yang lebih manual dibandingkan pendekatan deklaratif.

Opini pribadi:

Saya pikir daripada memikir bahwa satu bahasa diatas satunya, lebih penting untuk memikirkan program yang akan dibuat, serta kebutuhannya. Jika menggunakan UI, serahkan kepada deklaratif, dan jika perlu kontrol atas alur seperti membuat OS, maka serahkan kepada imperatif. Saya sendiri lebih memilih fungsional dibanding imperatif karena saya cenderung membuat program yang berkaitan dengan sub-routine, atau di Kotlin dikenal sebagai coroutines, jika saya lebih suka membuat program sistem maka pasti saya akan cenderung memilih imperatif.

2. Apa yang membedakan OOP pada kotlin dengan OOP pada java? *

"A lot of the standard boilerplate of object-oriented languages, such as getters, setters, and the logic for assigning constructor parameters to fields, is implicit in Kotlin and doesn't clutter your source code" [1] - Chapter 1.4.2

Paragraf tersebut menjelaskan bahwa Kotlin mengurangi boilerplate seperti *getter*, *setter*, dan *assignment constructor*, sehingga kode menjadi lebih ringkas dan tidak dipenuhi detail teknis. Hal ini memungkinkan *class* lebih fokus pada tanggung jawab utamanya, sehingga mendukung SRP.

"Kotlin's classes and interfaces differ a bit from what you might be used to from Java: for example, interfaces can contain property declarations. Kotlin's declarations are final and public by default. In addition, nested classes aren't inner by default: they don't contain an implicit reference to their outer class" [1] - Chapter 4

Di sini dapat dipahami bahwa dengan menjadikan *class* final secara *default*, Kotlin mendorong desain yang lebih tertutup terhadap modifikasi langsung dan hanya dapat diperluas secara eksplisit, yang sejalan dengan OCP, dan karena *interfaces can contain property declarations*, bisa dibilang bahwa *interface* bisa lebih spesifik dan ekspresif, sesuai dengan ISP.

The override modifier is used to mark methods and properties that override those from the superclass or interface. Unlike Java, which uses the @Override annotation, using the override modifier is mandatory in Kotlin. [1] - Chapter 4.1.1

Kutipan ini memberi tahu bahwa penggunaan *modifier override* membuat proses *overriding* lebih eksplisit dan aman, sehingga membantu menjaga konsistensi perilaku *subclass* terhadap *superclass* sesuai LSP.

Classes in Kotlin can't have static members. In fact, Kotlin doesn't have a static keyword like Java does. Instead, Kotlin relies on package-level functions (which can replace static methods in many situations) and object declarations (which serve to replace static methods as well as static fields in other cases). [1] - Chapter 4.4.2

Bagian atas menjelaskan bahwa tidak adanya *static member*, Kotlin mendorong penggunaan fungsi dan objek yang lebih fleksibel dan mudah diuji, sehingga mengurangi ketergantungan langsung pada implementasi dan mendukung DIP.

3. Sebutkan, dan jelaskan mengenai SOLID Principles *

"The Single Responsibility Principle is about functions and classes, but it reappears in a different form at two more levels…. A module should be responsible to one, and only one, actor." [4] - Chapter 7

Kutipan di atas menjelaskan bahwa setiap bagian program, termasuk modul, sebaiknya hanya memiliki satu tanggung jawab dan melayani satu pihak saja. Dengan begitu, kode menjadi lebih sederhana, mudah dipahami, dan tidak mudah rusak ketika terjadi perubahan.

"The OCP is one of the driving forces behind the architecture of systems. The goal is to make the system easy to extend without incurring a high impact of change. This goal is accomplished by partitioning the system into components, and arranging those components into a dependency hierarchy that protects higher-level components from changes in lower-level components. In other words, the behavior of a software artifact ought to be extendible, without having to modify that artifact." [4] - Chapter 8

Paragraf ini menjelaskan bahwa OCP bertujuan agar sistem mudah dikembangkan tanpa harus mengubah kode yang sudah ada. Hal ini dilakukan dengan membagi sistem ke dalam komponen dan mengatur dependensinya, sehingga perubahan di bagian bawah tidak memengaruhi bagian yang lebih tinggi.

"The LSP can, and should, be extended to the level of architecture. A simple violation of substitutability, can cause a system's architecture to be polluted with a significant amount of extra mechanisms." [4] - Chapter 9

Bagian ini menjelaskan bahwa LSP adalah prinsip yang menyatakan bahwa suatu objek dari *subclass* harus dapat menggantikan objek dari *superclass* tanpa merusak perilaku program. Prinsip ini juga berlaku pada level arsitektur. Jika LSP dilanggar, sistem akan menjadi lebih kompleks karena perlu menambahkan berbagai mekanisme tambahan untuk mengatasi ketidaksesuaian tersebut. Sebaliknya, dengan mengikuti LSP, desain sistem tetap konsisten, sederhana, dan mudah dipelihara.

"If you take a step back and look at the root motivations of the ISP, you can see a deeper concern lurking there. In general, it is harmful to depend on modules that contain more than you need… This principle advises software designers to avoid depending on things that they don't use" [4] - Chapter 10 and Chapter 6 part III

Di sini dijelaskan bahwa ISP adalah prinsip yang menyarankan agar suatu komponen tidak bergantung pada hal-hal yang tidak digunakannya. Artinya, *interface* atau modul sebaiknya dibuat lebih kecil dan spesifik sesuai kebutuhan. Jika sebuah modul bergantung pada terlalu banyak hal yang tidak relevan, hal itu dapat menambah kompleksitas dan membuat sistem lebih sulit untuk dipelihara.

"The Dependency Inversion Principle (DIP) tells us that the most flexible systems are those in which source code dependencies refer only to abstractions, not to concretions. The code that implements high-level policy should not depend on the code that implements lowlevel details. Rather, details should depend on policies." [4] - Chapter 11 and Chapter 6 part III

Terakhir, kutipan atas menjelaskan bahwa DIP adalah prinsip yang menyatakan bahwa kode sebaiknya bergantung pada abstraksi, bukan pada implementasi konkret. Artinya, bagian sistem yang berisi logika utama tidak boleh bergantung pada detail teknis, tetapi justru sebaliknya.

4. Apa perbedaan antara Class dan Data Class pada Kotlin ?

"The idea of a class is to encapsulate data and code that works on that data into a single entity." [1] - Chapter 2.2.1

Kalimat diatas menerangkan bahwa *class* adalah bentuk paling umum, tidak punya *behavior* khusus dan dipakai untuk logika umum, kebutuhan data seperti *toString(), equals()* dll harus dibuat sendiri jika diperlukan. Sederhana, dan dapat digunakan untuk memegang data saja, tetapi sebaiknya menggunakan data *class*, yang lebih terspesialisasi dalam memegang data.

"If you want your class to be a convenient holder for your data, you need to override these methods: toString, equals, and hashCode. Usually, the implementations of those methods are straightforward, and IDEs like IntelliJ IDEA can help you generate them automatically and verify that they're implemented correctly and consistently. The good news is, you don't have to generate all of these methods in Kotlin. If you add the modifier data to your class, the necessary methods are automatically generated for you." [1] - Chapter 4.3.2

Data class adalah bentuk class yang khusus untuk memegang data, cukup menulis data sebelum class dan Kotlin akan otomatis membuat metode-metode seperti `toString()`, `equals()`, `hashCode()`, selain itu ada juga `copy()`, dan `componentN()`. Jadi bisa dibilang data *class* adalah *class* yang berisi method yang dibutuhkan untuk menyimpan data.

Contoh perbedaan dalam kode:

```
class Mahasiswa(val name: String, val age: Int, val NIM: String)
val m1 = Mahasiswa("Budi", 20, "123")
val m2 = Mahasiswa("Budi", 20, "123")
println(m1 == m2)
println(m1)
```

Hasilnya:

```
false
Mahasiswa@1a2b3c
```

Disini printnya salah, karena == membandingkan referensi objek, bukan isi, `toString()` nya default.

Sintaks yang sama:

```
val m1 = Mahasiswa("Budi", 20, "123")
val m2 = Mahasiswa("Budi", 20, "123")
```

Pada `data class Mahasiswa(val name: String, val age: Int, val NIM: String)` akan menghasilkan:

```
true
Mahasiswa(name=Budi, age=20, NIM=123)
```

Ini hanya contoh untuk `toString()`, tetapi untuk copy/destructuring juga tidak dapat dilakukan secara langsung oleh *class* biasa.

E. Tautan Git

Berikut adalah tautan untuk repository yang dibuat untuk source code:

https://github.com/SebastianAbeSantoso/MODUL-MOBILE/tree/main/MODUL%201

F. Daftar Pustaka

[1] R. E. S. I. D. J. Sebastian Aigner, "Kotlin in Action, Second Edition MEAP V09," 2024.

[2] Michael L. Scott, "In Praise of Programming Language Pragmatics,Third Edition," 2009. doi: https://doi.org/10.1016/B978-0-12-374514-9.X0001-8.

[3] React, "Reacting to Input with State." Accessed: Apr. 03, 2026. [Online]. Available: https://react.dev/learn/reacting-to-input-with-state

[4] R. C. Martin, "Clean Architecture: A Craftsman's Guide to Software Structure and Design," 2018.